

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1711

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour

Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

*I **B.Bhargavi 192311136** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*B.Bhargavi*

Reverse Engineering

1.Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

Problem Understanding:

A large e-commerce platform recommends products to users based on their previous purchases, searches, clicks, ratings, and browsing history. Since millions of users generate data continuously, the system needs distributed processing. PySpark and RDDs can process this large amount of data efficiently.

Reverse-Engineered Architecture

User Activity → Data Collection → PySpark → RDD Processing → Feature Extraction → Recommendation Algorithm → Intelligent Agent → Recommended Products

Data Collection

The system can collect:

- User ID
- Product ID
- Search history
- Click history
- Purchase history
- Product ratings
- Time spent on products
- Shopping-cart activity

The data can come from application logs, databases, APIs, and real-time events.

PySpark and RDD Processing

The collected data can be converted into an RDD.

```
data = sc.textFile("user_activity.csv")
```

Remove invalid records:

```
clean_data = data.filter(lambda x: x != "")
```

Extract user and product information:

```
user_products = clean_data.map(
    lambda x: (x.split(",")[0], x.split(",")[1])
)
```

Group products by user:

```
recommendations = user_products.groupByKey()
```

Count product interactions:

```
product_count = user_products.map(
    lambda x: (x[1], 1)
).reduceByKey(lambda a, b: a + b)
```

Intelligent Agent

An intelligent recommendation agent observes user behavior and decides which products should be recommended.

For example:

User searches for laptop → Agent identifies interest → Finds related products → Ranks products → Displays recommendations.

The agent continuously updates recommendations when new user activity is received.

Decision Making

The system can calculate a recommendation score using:

- Previous purchases
- Product similarity
- User ratings
- Recent searches
- Popular products

Higher-scoring products are recommended first.

Scalability and Fault Tolerance

PySpark distributes RDD processing across multiple machines. RDDs are fault tolerant because Spark can reconstruct lost partitions using their lineage

Expected Output

The user receives personalized products such as:

"Recommended for you: Laptop, Wireless Mouse, Laptop Bag"

Conclusion

The reverse-engineered system uses PySpark and RDDs for scalable data processing and intelligent agents for dynamic recommendation decisions. This architecture allows an e-commerce platform to process millions of user interactions efficiently.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

Problem Understanding

A smart traffic management system automatically changes traffic signals according to vehicle density and congestion. The system must make decisions quickly because traffic conditions change continuously.

Reverse-Engineered Architecture

Traffic Sensors/Cameras → Real-Time Data → PySpark → RDD Processing → Traffic Analysis → Intelligent Traffic Agent → Signal Decision → Traffic Lights

Data Collection

The system receives:

- Number of vehicles
- Vehicle speed
- Lane occupancy
- Traffic density
- Waiting time
- Accident information
- Traffic signal status

Sensors and cameras continuously generate data.

RDD-Based Processing

Traffic data can be loaded into an RDD:

```
traffic = sc.textFile("traffic_data.csv")
```

Remove invalid data:

```
clean = traffic.filter(lambda x: x != "")
```

Extract road and vehicle information:

```
road_data = clean.map(
    lambda x: (x.split(",")[0], int(x.split(",")[1]))
)
```

Calculate total vehicles:

```
vehicle_count = road_data.reduceByKey(lambda a, b: a + b)
```

Filter highly congested roads:

```
congested = vehicle_count.filter(
    lambda x: x[1] > 100
)
```

Intelligent Agents

Each traffic junction can be controlled by an intelligent agent.

For example:

Agent A → Junction 1

Agent B → Junction 2

Agent C → Junction 3

Agents communicate traffic conditions and coordinate signal timings.

Decision Making

Suppose:

- Road A = 120 vehicles
- Road B = 40 vehicles
- Road C = 20 vehicles

The intelligent agent identifies Road A as highly congested and increases its green-light duration.

For example:

Normal green time = 60 seconds

High congestion = 90 seconds

Time Constraints

Traffic decisions must be made within seconds. Therefore:

- Data processing must be fast.
- Only relevant data should be processed.
- Distributed processing reduces computation time.
- Agents make local decisions quickly.

Fault Tolerance

If one Spark worker fails, RDD lineage allows lost partitions to be recomputed.

Expected Output

The system dynamically changes traffic signals:

High congestion → Longer green signal

Low congestion → Shorter green signal

Conclusion

The reverse-engineered traffic system combines PySpark RDD processing with intelligent agents to analyze traffic conditions and dynamically control signals. This reduces congestion and improves traffic flow.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

Problem Understanding

A banking system needs to identify fraudulent transactions within milliseconds. Since banks process millions of transactions, distributed AI processing is required.

Reverse-Engineered Architecture

Transaction → Data Collection → PySpark → RDD → Feature Extraction → Anomaly Detection → Fraud Agent → Alert → Bank/Customer

Data Collection

The system can collect:

- Transaction ID
- Customer ID
- Transaction amount
- Location
- Time
- Device information
- Merchant information
- Previous transaction history

RDD Processing

Load transaction data:

```
transactions = sc.textFile("transactions.csv")
```

Remove invalid transactions:

```
clean = transactions.filter(lambda x: x != "")
```

Extract transaction amount:

```
amounts = clean.map(  
    lambda x: float(x.split(",")[2])  
)
```

Find high-value transactions:

```
high_value = amounts.filter(lambda x: x > 100000)
```

Group transactions by customer:

```
customer_data = clean.map(  
    lambda x: (x.split(",")[1], x)
```

).groupByKey()

Feature Extraction

The system can generate features such as:

- Transaction amount
- Transaction frequency
- Distance from previous location
- Unusual transaction time
- New device
- Unusual merchant
- Previous fraud history

Anomaly Detection

The AI model compares the current transaction with normal customer behavior.

Example:

Normal customer transaction = ₹2,000–₹5,000

Suddenly:

New transaction = ₹2,00,000 from another country

The system assigns a high fraud probability.

Intelligent Fraud Agent

The fraud detection agent receives the model's result.

Example:

Transaction → AI model → Fraud probability = 95% → Agent → Block/Alert

The agent can:

- Allow transaction
- Request additional verification
- Temporarily block transaction
- Send alert
- Notify security team

Decision Making

A simple decision rule can be:

Fraud Score < 30% → Allow

30–70% → Verify

>70% → Alert/Block

Distributed Processing

PySpark allows transactions to be processed across multiple worker nodes. This is useful for large banking datasets.

Expected Output

Example:

Transaction ID: TX1023

Fraud Probability: 95%

Status: Suspicious

Action: Transaction blocked and customer alerted

Conclusion

The reverse-engineered fraud detection pipeline uses PySpark RDDs for distributed transaction processing, AI models for anomaly detection, and intelligent agents for rapid response. This enables banks to detect suspicious transactions with low latency.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

Problem Understanding

An online learning platform changes the difficulty of learning materials according to student performance and engagement.

For example:

Student performs well → System provides advanced questions.

Student struggles → System provides easier questions and additional explanations.

Reverse-Engineered Architecture

Student Activity → Data Collection → PySpark → RDD Processing → Performance Analysis → Intelligent Learning Agent → Content Selection → Student Feedback

Data Collection

The system collects:

- Student ID
- Quiz scores
- Assignment marks
- Time spent
- Number of attempts
- Completed lessons
- Wrong answers
- Video watching behavior
- Learning frequency

RDD Processing

Load learner data:

```
students = sc.textFile("student_data.csv")
```

Remove invalid records:

```
clean = students.filter(lambda x: x != "")
```

Extract student marks:

```
marks = clean.map(  
    lambda x: (x.split(",")[0], int(x.split(",")[1]))  
)
```

Calculate average score:

```
average = marks.groupByKey().mapValues(  
    lambda x: sum(x) / len(x)  
)
```

Find students requiring support:

```
weak_students = average.filter(  
    lambda x: x[1] < 50  
)
```

Intelligent Learning Agent

The intelligent agent observes student performance and chooses suitable learning content.

For example:

Score > 80% → Advanced content

Score 50–80% → Normal content

Score < 50% → Basic content + revision

Adaptive Feedback

Suppose a student repeatedly makes mistakes in Python loops.

The agent can recommend:

1. Python loop explanation
2. Simple examples
3. Practice questions
4. Quiz
5. Advanced exercises after improvement

Decision Making

The agent considers:

- Academic performance
- Learning speed
- Engagement
- Previous mistakes
- Attempt history

It then selects the next learning activity.

Fault Tolerance and Scalability

Large educational platforms may have millions of learner records. PySpark distributes the processing across multiple machines, while RDDs provide fault tolerance.

Expected Output

Example:

Student Score: 42%

Detected Area: Python Loops

Recommendation: Basic Loop Tutorial + Practice Quiz

Conclusion

The reverse-engineered personalized learning system uses PySpark RDDs to analyze large amounts of student data and intelligent agents to dynamically personalize educational content. This improves learning efficiency and provides individualized feedback.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

Problem Understanding

Cloud platforms contain thousands of servers and applications that continuously generate logs. The monitoring system must detect anomalies and automatically scale resources when necessary.

Reverse-Engineered Architecture

Cloud Resources → Log Collection → PySpark → RDD Processing → Feature Extraction → Anomaly Detection → Intelligent Agents → Scaling/Response

Data Collection

The system can collect:

- CPU usage
- Memory usage
- Network traffic
- Server logs
- Application errors

- Response time
- Number of requests
- Disk usage

PySpark RDD Processing

Load cloud logs:

```
logs = sc.textFile("cloud_logs.txt")
```

Remove empty records:

```
clean_logs = logs.filter(lambda x: x != "")
```

Extract server and CPU information:

```
cpu_data = clean_logs.map(  
    lambda x: (x.split(",")[0],  
               float(x.split(",")[1]))  
)
```

Calculate average CPU usage:

```
avg_cpu = cpu_data.groupByKey().mapValues(  
    lambda x: sum(x) / len(x)  
)
```

Find servers with high CPU:

```
high_cpu = avg_cpu.filter(  
    lambda x: x[1] > 80  
)
```

Anomaly Detection

The AI system identifies unusual patterns.

Example:

Normal CPU = 30–60%

Warning = 60–80%

Critical = >80%

If CPU suddenly reaches 95%, the system identifies a possible anomaly or heavy workload.

Intelligent Agents

Different intelligent agents can perform different tasks:

Monitoring Agent

- Observes system metrics.

Anomaly Detection Agent

- Identifies unusual behavior.

Scaling Agent

- Adds or removes cloud resources.

Response Agent

- Performs corrective actions.

Agent Interaction

Example:

Monitoring Agent

↓

CPU = 95%

↓

Anomaly Agent



High Load Detected



Scaling Agent



Create Additional Server



Response Agent



System Stabilized

Automatic Scaling

If the workload increases:

$\text{CPU} > 80\% \rightarrow \text{Add resources}$

If workload decreases:

$\text{CPU} < 30\% \rightarrow \text{Reduce unnecessary resources}$

This improves both performance and cost efficiency.

RDD Fault Tolerance

RDDs are immutable and distributed. If a worker node fails, Spark can reconstruct lost partitions using RDD lineage.

This makes the monitoring system more reliable.

Expected Output

Example:

Server: Cloud-Server-04

CPU: 95%

Status: Critical

Action: Scaling initiated

New Server: Created

Conclusion

The cloud monitoring system combines PySpark, RDD-based distributed log processing, anomaly detection, and intelligent agents. The agents coordinate monitoring, scaling, and corrective actions, allowing the cloud platform to automatically respond to changing workloads.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation
-----------------	----	--	-------------------------------------	------------------------------------	-------------------------------------